CSCI 6461 | TOPIC 5

Memory Architecture & Addressing Principles

The load/store model, address computation, data sizing, and memory hierarchy interactions in AArch64

Dr. J. Burns
Computer Systems Architecture — AArch64 Reference Platform

Topic 5: High-Level Module Roadmap

1. **The Memory/Compute Divide in Architecture**
   - Why RISC separates memory transfers from arithmetic operations.
   - Bridging register speed with memory capacity.
2. **Data Types, Widths, and Conversions**
   - Handling bytes, halfwords, words, and doublewords cleanly.
   - Understanding when sign-extension matters vs. zero-extension.
3. **How Processors Calculate Memory Addresses**
   - Offsets, array indexing, and pointer increment patterns.
   - Position-independent memory references for global variables.
4. **Memory Performance, Alignment, & Stack Organization**
   - Moving double-word pairs efficiently with `ldp` and `stp`.
   - Alignment rules and managing runtime function stack frames.

PART 1

The Load/Store Architectural Boundary

The Fundamental Memory Challenge

- Register access is extremely fast, while off-chip DRAM access is much slower; exact latencies are implementation-dependent.
- **Architectural Question:** How should the instruction set organize data movement between memory and the register file?
- AArch64 uses a strict **load/store discipline**: ALU instructions operate on registers, while explicit load/store instructions access memory.
- Caches, prefetching, speculation, and out-of-order execution are microarchitectural techniques used to reduce the impact of memory latency.

The Architectural Rule

ALU instructions process *only* registers. Memory access is performed by load/store-family instructions.

Separating Computation from Data Movement

The Pipeline Contract

- Arithmetic instructions do not directly access data memory.
- Simplifies datapath organization and operand handling.
- Makes memory operations explicit for scheduling and dependency analysis.

Three-Step Execution Cycle

1. **Load:** Fetch data into registers.
2. **Compute:** Perform ALU operations.
3. **Store:** Write result back if needed.

Comparing Memory Models: RISC vs. CISC

AArch64 (Explicit Load/Store)

```
ldr x1, [x0]       // Load from address in x0
add x2, x2, x1     // Compute in registers
str x2, [x0]       // Write result back
```

- 3 simple, orthogonal instructions.
- Compiler can schedule other work during memory load latency.

x86-64 (Register-Memory CISC)

```
add [rdi], rax     // Memory Read-Modify-Write
```

- 1 complex instruction.
- Hardware decoder may decode into one or more internal micro-operations depending on implementation.

Takeaway: AArch64 exposes data movement through explicit load/store instructions, while x86-64 permits some architectural instructions to combine memory access with computation.

Addressing Memory: Bytes, Words, and Doublewords

- Memory is a linear sequence of **8-bit bytes**, accessed via 64-bit virtual addresses (though implemented virtual/physical address widths are smaller).
- The instruction/access size determines the transfer width:

Data Type	Width	Load Syntax	Store Syntax
Byte	8-bit	`ldrb wt, [xn]`	`strb wt, [xn]`
Halfword	16-bit	`ldrh wt, [xn]`	`strh wt, [xn]`
Word	32-bit	`ldr wt, [xn]`	`str wt, [xn]`
Doubleword	64-bit	`ldr xt, [xn]`	`str xt, [xn]`

The base register (xn) holds the 64-bit memory pointer enclosed in brackets: `[xn]`.

PART 2

Data Sizing & Extension Rules

The Sub-Word Loading Problem

- Registers are 64 bits wide, but data items in memory are often 8 bits or 16 bits.
- When an 8-bit byte is loaded into a 64-bit register, **what happens to the remaining 56 bits?**
- The answer depends on whether the data is **unsigned** (e.g., characters, raw bytes) or **signed** (e.g., small integers).

Unsigned Data (Zero-Extension)

Fill all upper bits with `0`s to preserve magnitude.

Signed Data (Sign-Extension)

Replicate the sign bit to preserve negative values.

Byte, Halfword, and 32-Bit Word Loads

- Byte and halfword loads such as `ldrb` and `ldrh` zero-extend into `wt`; a 32-bit `ldr wt` writes `wt`, which clears the upper 32 bits of `xt`.

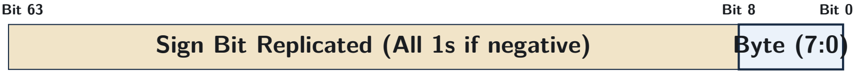
Bit 63	Bit 8	Bit 0
Automatically Cleared to Zeros (Bits 63:8)	**Byte (7:0)**	

Example: Loading Byte `0x80`

`ldrb w0, [x1]` $\Longrightarrow x_0 =$ `0x00000000_00000080` $= +128_{10}$ (Unsigned integer)

Signed Loads: Sign-Extension (`ldrsb`, `ldrsh`, `ldrsw`)

- The `ldrsb`, `ldrsh`, and `ldrsw` variants load a smaller signed value and sign-extend it to the destination width.
- Sign extension copies the source sign bit across the upper destination bits.



Example: Loading Byte `0x80` (-128 in 8-bit signed)

`ldrsb x0, [x1]` $\Longrightarrow x_0 =$ `0xffffffff_ffffff80` $= -128_{10}$ (Sign preserved in 64 bits)

Sign vs. Zero Extension: Side-by-Side

Memory Byte at Address `x1` = `0x80` (Binary: `1000 0000`)

Executed Instruction	64-bit Hex in `x0`	Decimal Value and Meaning
`ldrb w0, [x1]`	`0x00000000_00000080`	$+128$ (Unsigned zero-extension)
`ldrsb w0, [x1]`	`0x00000000_ffffff80`	-128 (Sign-extended to 32b, upper zeroed)
`ldrsb x0, [x1]`	`0xffffffff_ffffff80`	-128 (Sign-extended to full 64b)

Common Bug

Using `ldrb` on a signed negative variable results in a large positive number, causing if-statements and comparisons to fail unexpectedly.

Storing Data: Truncation Rules

- Stores move data from a register into memory.
- The instruction width determines how many bytes from the register are written:

Instruction	Bits Read from Register	Bytes Written to Memory
`strb wt, [xn]`	Bits [7:0]	Writes exactly 1 byte
`strh wt, [xn]`	Bits [15:0]	Writes exactly 2 bytes
`str wt, [xn]`	Bits [31:0]	Writes exactly 4 bytes
`str xt, [xn]`	Bits [63:0]	Writes full 8 bytes (doubleword)

Takeaway: Storing a byte or halfword modifies *only* the targeted memory locations without overwriting neighboring bytes.

PART 3

AArch64 Addressing Modes

What Is an Addressing Mode?

- An **addressing mode** is the method used by an instruction to compute the **Effective Address (EA)** of a data operand in memory.
- Different data structures require different address calculations:
  - Fixed struct members $\rightarrow$ **Base + Constant Offset**
  - Sequential arrays $\rightarrow$ **Auto-Incrementing Pointers**
  - Array indexing (`arr[i]`) $\rightarrow$ **Base + Scaled Register Index**
  - Global variables $\rightarrow$ **PC-Relative Page Addressing**

Mode 1: Base Register with Immediate Offset

- Adds a fixed constant to a base pointer without modifying the base register.
- **Syntax:** `ldr xt, [xn, #offset]` $\implies$ EA $= x_n + \text{offset}$.

Base Address (`x0`) → +Offset → **Target Memory Address**

Example: Reading Struct Fields

```
ldr x1, [x0, #0]     // Read field 1 at offset 0
ldr x2, [x0, #8]     // Read field 2 at offset 8
ldr w3, [x0, #16]    // Read 32-bit field at offset 16
```

Unsigned Scaled Immediate Offsets

- In 32-bit instructions, immediate fields have limited bit width.
- In the unsigned scaled-immediate form, AArch64 scales the 12-bit offset by the access size:
  - 64-bit loads scale offsets by $\times 8$ $(0, 8, 16, \ldots, 32760)$.
  - 32-bit loads scale offsets by $\times 4$ $(0, 4, 8, \ldots, 16380)$.
- Allows instructions to reach up to ≈ 32 KB from a base pointer using only 12 encoding bits.

Signed Unscaled Offsets (`ldur`/`stur`)

For signed unscaled immediate offsets (which do not scale by operand size and can be negative), use `ldur xt, [xn, #-8]`.

Mode 2: Pre-Indexed Addressing (Update Before Access)

- Updates the base register **before** reading or writing memory.
- **Syntax:** `ldr xt, [xn, #offset]!` (Exclamation mark indicates update).

Step-by-Step Action

1. $x_n \leftarrow x_n + \text{offset}$ (Update pointer)
2. Access memory at the **new** address in x_n.

Primary Use Case: Stack Push

```
str x30, [sp, #-16]!
```

Moves the stack pointer down by 16 bytes, then writes register x30 to the new top of stack.

Mode 3: Post-Indexed Addressing (Update After Access)

- Reads or writes memory at the current address, then updates the base pointer **afterwards**.
- **Syntax:** `ldr xt, [xn], #offset` (Offset written outside the brackets).

Step-by-Step Action

1. Access memory at the **current** address in x_n.
2. $x_n \leftarrow x_n + \text{offset}$ (Advance pointer)

Primary Use Case: Array Stepping & Stack Pop

```
ldr x0, [x1], #8
```

Loads data from current address in x1, then automatically advances x1 to the next 8-byte element.

Comparing the Three Offset Modes

Mode	Syntax Pattern	Address Accessed	Base Register After
Standard Offset	`[xn, #8]`	$x_n + 8$	x_n (Unchanged)
Pre-Indexed	`[xn, #8]!`	$x_n + 8$	$x_n + 8$ (Updated)
Post-Indexed	`[xn], #8`	x_n	$x_n + 8$ (Updated)

Assume `x1 = 0x1000` Initially

`ldr x0, [x1, #8]` $\Longrightarrow$ Reads from `0x1008`; x_1 remains `0x1000`

`ldr x0, [x1, #8]!` $\Longrightarrow$ Reads from `0x1008`; x_1 becomes `0x1008`

`ldr x0, [x1], #8` $\Longrightarrow$ Reads from `0x1000`; x_1 becomes `0x1008`

Mode 4: Register Offset with Scaling

- Uses an index register (x_m) multiplied by element size to access array items:

```
ldr xt, [xn, xm, lsl #shift]
```

- The instruction encodes the scaled register offset directly, though implementation latency depends on the processor.

Array Element Type	**Assembly Instruction**	**Effective Address**
Byte (`uint8`)	`ldrb w0, [x1, x2]`	$\text{EA} = x_1 + x_2$
Halfword (`uint16`)	`ldrh w0, [x1, x2, lsl #1]`	$\text{EA} = x_1 + (x_2 \times 2)$
Word (`uint32`)	`ldr w0, [x1, x2, lsl #2]`	$\text{EA} = x_1 + (x_2 \times 4)$
Doubleword (`uint64`)	`ldr x0, [x1, x2, lsl #3]`	$\text{EA} = x_1 + (x_2 \times 8)$

Directly implements high-level array access: `val = arr[i];`.

Mixing 32-bit Indices with 64-bit Pointers

- In C code, loop counter variables are frequently 32-bit integers (`int i`).
- AArch64 address calculations use 64-bit general-purpose registers.
- AArch64 can extend and scale a 32-bit index register in one step:

Signed vs. Unsigned Indexing

```
ldr x0, [x1, w2, sxtw #3]    // Sign-extend 32-bit index w2, multiply by 8, add to x1
ldr x0, [x1, w2, uxtw #3]    // Zero-extend 32-bit index w2, multiply by 8, add to x1
```

Benefit: Eliminates separate instructions to convert `int` indices to 64-bit pointers.

Mode 5: PC-Relative Global Variable Access

- Position-Independent Code (PIC) allows code to load anywhere in memory.
- Global variables are often addressed relative to the current **Program Counter (pc)**, though GOT-based sequences may also be used.
- A common page-relative pattern involves a two-step sequence:

Loading a Global Variable (Common Pattern)

```
adrp x0, my_global                // Step 1: Form base address of the 4KB page
ldr x1, [x0, #:lo12:my_global]    // Step 2: Add page offset and load value
```

The `adrp` instruction forms a page-relative address within its architectural range; the linker then resolves the page and low-offset relocation for the selected code model.

PC-Relative Literal Pools

- How do we load arbitrary 64-bit constants that cannot fit in an instruction?
- `ldr x0, =constant` is an assembler pseudo-instruction; the assembler may choose a literal pool or another constant-materialization sequence.

Assembly Syntax

```
ldr x0, =0x0123456789abcdef   // Pseudo-instruction: assembler generates constant loading sequence
```

If a literal pool is selected, the instruction performs a PC-relative load from the nearby constant.

PART 4

Multi-Register Transfers, Alignment, & Stack

Paired Transfers: `ldp` and `stp`

- AArch64 provides dedicated instructions to transfer **two registers with one architectural instruction**:

```
ldp x0, x1, [x2]    stp x0, x1, [x2]
```

- Reduces instruction count by handling two registers per memory transfer instruction.

Instruction	Action	Primary Use Case
`stp x29, x30, [sp, #-16]!`	Push Frame Pointer & Link Register	Function entry (prologue)
`ldp x29, x30, [sp], #16`	Restore FP & LR, adjust stack	Function exit (epilogue)
`ldp x0, x1, [x2]`	Load two 64-bit values into x0, x1	Adjacent-value or buffer a

Why Paired Transfers Matter

Individual Transfers

```
str x29, [sp, #-8]!
str x30, [sp, #-8]!
```

- Requires 2 separate instructions.
- Higher instruction footprint.

Paired Transfer

```
stp x29, x30, [sp, #-16]!
```

- Uses a single instruction.
- Stores two registers with one instruction.

Instruction-Count Impact: For the shown pair of saves/restores, a paired transfer uses one memory instruction instead of two.

Memory Alignment Principles

- An access is **naturally aligned** if its memory address is an exact multiple of the data size:

$$\text{Address} \pmod{\text{Data Size}} = 0$$

Aligned Access

64-bit Word [Bytes 0–7] Typically maps cleanly to memory access structures without boundary crossing.

Unaligned Access

Cache Line 1 | Cache Line 2 May require additional internal accesses, especially when crossing cache-line or page boundaries.

Many ordinary accesses to Normal memory can be unaligned, although boundary crossings may require extra internal work. Some access types or configured environments impose stricter alignment requirements.

Stack Alignment: An AAPCS64 ABI Requirement

- The standard AArch64 procedure call standard (AAPCS64) mandates a strict **16-byte alignment rule for the Stack Pointer (sp)** as a core ABI requirement.
- Violating the 16-byte alignment rule breaks the ABI contract and can cause incorrect interaction with code that assumes a conforming stack layout.
- When allocating local variables or saving registers, stack-frame allocation is normally rounded to a multiple of 16 bytes.

Correct Stack Adjustment Example

```
sub sp, sp, #16   // Correct: 16-byte multiple
sub sp, sp, #32   // Correct: 32-byte multiple
sub sp, sp, #8    // Violation of AAPCS64 16-byte ABI requirement
```

Standard Function Stack Frame Layout

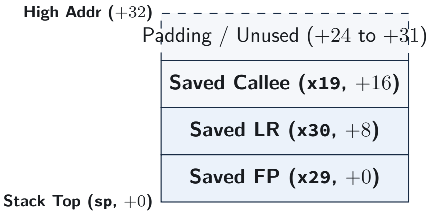
Function Code

```
my_func:
    // 1. Save Frame Pointer & Link Reg
    // Allocate 32 bytes total
    stp  x29, x30, [sp, #-32]!
    mov  x29, sp

    // 2. Save callee-saved register
    str  x19, [sp, #16]

    // ... Function body ...

    // 3. Restore and return
    ldr  x19, [sp, #16]
    ldp  x29, x30, [sp], #32
    ret
```



Worked Pattern 1: Stepping Through an Array

Goal: Sum N 64-bit Integers

AArch64 Implementation (Using Post-Indexed Addressing)

```
// x0 = array pointer, x1 = count n
sum_loop:
        mov x2, xzr          // sum = 0
        cbz x1, .Ldone       // Exit if count == 0
.Lbody:
        ldr x3, [x0], #8     // Load element AND increment pointer by 8
        add x2, x2, x3       // sum += element
        sub x1, x1, #1       // count– (flags not needed)
        cbnz x1, .Lbody      // Loop if count != 0
.Ldone:
        mov x0, x2           // Return sum in x0
        ret
```

Post-indexing avoids a separate pointer-update instruction such as `add x0, x0, #8` inside the loop.

Worked Pattern 2: Indexed Array Search

Goal: Check Element at Index i in Array

AArch64 Implementation (Using Scaled Register Offset)

```
// x0 = array pointer, x1 = index i, x2 = target value
find_element:
        ldr x3, [x0, x1, lsl #3]    // Load arr[i] using Base + (i * 8)
        cmp x3, x2                  // Compare arr[i] with target
        cset w0, eq                 // w0 = 1 if match, else 0
        ret
```

Combines index scaling and load into a single instruction without modifying the base pointer `x0`.

Worked Pattern 3: Copying a Data Block

Goal: Copy 32 Bytes from Source to Destination

AArch64 Implementation (Using Paired Transfers)

```
// x0 = destination pointer, x1 = source pointer
copy_32_bytes:
        ldp x2, x3, [x1]         // Load first 16 bytes (two doublewords)
        ldp x4, x5, [x1, #16]    // Load next 16 bytes
        stp x2, x3, [x0]         // Store first 16 bytes
        stp x4, x5, [x0, #16]    // Store next 16 bytes
        ret
```

Copies 32 bytes using four paired load/store instructions.

Addressing Mode Selection Guide

Programming Scenario	Best Addressing Mode	Example Instruction
Accessing Struct Field	Base + Immediate Offset	`ldr x1, [x0, #8]`
Pushing to Stack	Pre-Indexed Auto-Update	`str x30, [sp, #-16]!`
Popping from Stack	Post-Indexed Auto-Update	`ldr x30, [sp], #16`
Sequential Array Stepping	Post-Indexed Auto-Update	`ldr x2, [x0], #8`
Random Array Index (`a[i]`)	Scaled Register Offset	`ldr x2, [x0, x1, lsl #3]`
Global Variable Reference	PC-Relative Page Offset	`adrp` + `ldr`

Hardware Memory Access: The Cache Line View

- Hardware fetches memory in blocks known as **Cache Lines** (implementation-dependent; 64 bytes is common).
- **Spatial Locality:** Accessing contiguous memory (e.g., sequential array stepping) often improves spatial locality and prefetch effectiveness.
- Choosing sequential addressing modes helps processor hardware prefetchers predict memory access patterns accurately.

Performance Insight

Sequential access patterns usually make better use of spatial locality and hardware prefetching than scattered access patterns.

Common Memory Access Mistakes

- **Wrong Sizing:** Using `ldr` (64-bit) instead of `ldr wt` (32-bit) on an array of integers reads unintended adjacent bytes as part of the value.
- **Missing Sign Extension:** Using `ldrb` instead of `ldrsb` on negative numbers corrupts signed comparisons.
- **Incorrect Shift Scale:** Using `lsl #2` instead of `lsl #3` on an array of 64-bit pointers accesses the wrong memory location.
- **Stack Misalignment:** Violating AAPCS64 16-byte stack alignment conventions can lead to compliance or runtime issues.

Review and Self-Test Questions

Topic 5: Comprehensive Summary

- **Load/Store Discipline:** ALU instructions operate on registers, while load/store-family instructions perform memory transfers.
- **Data Widths & Extensions:** Sub-word loads zero-extend by default; signed data requires explicit `ldrsb`, `ldrsh`, or `ldrsw` to preserve negative values.
- **Core Addressing Modes:**
  - Standard offset (`[xn, #imm]`): Base remains unchanged.
  - Pre-indexed (`[xn, #imm]!`): Base updated before access (push).
  - Post-indexed (`[xn], #imm`): Base updated after access (pop / loop step).
  - Scaled register offset (`[xn, xm, lsl #3]`): base-plus-scaled-index addressing in one instruction.
- **Paired Transfers & Stack Layout:** `ldp`/`stp` reduce instruction count for multi-register transfers; AAPCS64 mandates 16-byte stack alignment.

Self-Test Questions: Sizing & Addressing Modes

1. **Sign Extension vs. Zero Extension:** Memory byte at address `0x2000` holds value `0xfe` (-2 in signed 8-bit). What value resides in register `x0` after:
   - `ldrb w0, [x1]` (where $x_1 =$ `0x2000`)
   - `ldrsb x0, [x1]` (where $x_1 =$ `0x2000`)
2. **Addressing Mode Mechanics:** If register `x1` holds address `0x3000`, what address is read and what is the final value in `x1` for:
   - `ldr x0, [x1, #8]`
   - `ldr x0, [x1, #8]!`
   - `ldr x0, [x1], #8`
3. **Array Element Indexing:** Given base array pointer in `x0` and index i in `x1`, write a single instruction to load a 32-bit integer `arr[i]` into `w2`.

Self-Test Questions: Stack Alignment & Paired Transfers

1. **Paired Stack Operation:** Explain what happens during the instruction:

   ```
   stp x29, x30, [sp, #-16]!
   ```

   How does `sp` change, and where are `x29` and `x30` stored?

2. **Stack Alignment Enforcement:** Why does the AAPCS64 ABI require the stack pointer to be 16-byte aligned even when storing a single 8-byte register?

3. **Position-Independent References:** Why are `adrp` and offset/GOT sequences commonly used for position-independent global references in AArch64?

Looking Ahead

Next Lecture Preview

Our Next Topic: AArch64 Subroutines, Calling Conventions, & the Stack

- **Subroutine Calls:** Using `bl`, `blr`, and `ret`.
- **Calling Convention:** Passing arguments, returning values, and preserving registers.
- **Stack Frames:** Building and restoring function stack frames using AAPCS64 conventions.